CMAS4G2: A Locally Deployed Cooperative Multi-Agent System for Gherkin Acceptance Test Generation

Donghao Huang[1,2], Shila Chew[3], and Zhaoxia Wang[1] ⋆

[1] School of Computing and Information Systems, Singapore Management University, Singapore, Singapore
dh.huang.2023@engd.smu.edu.sg, zxwang@smu.edu.sg
[2] Research and Development, Mastercard, Arlington, VA, USA
donghao.huang@mastercard.com
[3] Research and Development, Mastercard, Singapore, Singapore
shila.chew@mastercard.com

Abstract. Acceptance test automation remains a persistent bottleneck in software engineering, as translating natural-language requirements into executable specifications demands extensive expertise and manual effort. While large language models (LLMs) show promise in code generation, their integration into Behavior-Driven Development (BDD) pipelines—particularly for Gherkin test authoring—remains underexplored. We introduce **CMAS4G2**, a *Cooperative Multi-Agent System for Gherkin Generation* built on Microsoft's AutoGen Swarm framework. CMAS4G2 coordinates specialized Validator, Quality-Assurance, and Supervisor agents to automate requirement validation and acceptance-test synthesis. Generated artifacts are evaluated at scale using the LLM-as-a-Judge (LAJ) framework [5], which provides rubric-driven, automated coverage assessment aligned with expert human judgments. Experiments on a 200-ticket benchmark derived from the Kill Bill platform show that open-weight models deployed locally on consumer hardware approach proprietary performance: GPT-OSS 20B (medium reasoning, our best-performing open-weight configuration by overall success) attains 87.6% overall success and 85.4% coverage versus Claude Sonnet 4's 90.4% and 91.6%. The low-reasoning variant achieves marginally higher coverage (85.7%) at slightly lower overall success (86.6%). These results demonstrate the feasibility of the proposed CMAS4G2, showing that cooperative, open-weight LLM agents can deliver competitive acceptance-test automation without cloud dependence, enabling cost-efficient, privacy-preserving quality assurance.

Keywords: Multi-Agent Systems · Acceptance Testing · Gherkin · Large Language Models · Behavior-Driven Development

⋆ Corresponding author: zxwang@smu.edu.sg

1 Introduction

Automated acceptance testing remains difficult: translating high-level requirements into executable tests demands domain expertise, sustained effort, and careful maintenance [18]. Traditional tools struggle to bridge the semantic gap between natural-language requirements and testable specifications [2]. Large language models (LLMs) show promise in code generation and semantic translation, but their application to Behavior-Driven Development (BDD)–particularly Gherkin–is underexplored [15]. In order to address these gaps, we propose CMAS4G2, a *Cooperative Multi-Agent System for Gherkin Generation*. CMAS4G2 uses Microsoft's AutoGen Swarm [10] to coordinate specialized agents for requirement validation, test generation, and quality assurance, enforcing structured handoffs and internal checks. To measure outcome quality at scale, we adopt the *LLM-as-a-Judge* (LAJ) framework [5], which provides rubric-driven, automated coverage assessment aligned with expert judgments while enabling high-throughput evaluation. On a 200-ticket benchmark, we compare open-weight models (e.g., Qwen3, GPT-OSS) optimized for local deployment with a proprietary baseline (Claude Sonnet 4), demonstrating that local models achieve competitive success rates while revealing critical trade-offs between reasoning intensity and multi-agent coordination stability. Our main contributions are:

1. **Architecture:** Propose CMAS4G2, a novel cooperative multi-agent framework for automated Gherkin acceptance test generation using AutoGen Swarm.
2. **Empirical findings:** Provide systematic evidence that open-weight, locally deployable models approach proprietary performance (e.g., GPT-OSS 20B: 87.6% overall success, 85.4% coverage vs. Claude Sonnet 4: 90.4%, 91.6%), and demonstrate that reasoning-mode configuration critically affects generation quality and coverage.
3. **Open-source release:** Publicly release code, datasets, and evaluation results upon acceptance to ensure reproducibility and facilitate further research.[4]

2 Related Work

LLM-based test generation has progressed rapidly, particularly in the context of unit testing. Industrial-scale deployments such as Meta's TestGen-LLM have reported build success rates of 75% and test pass rates of 57% across large production codebases [1]. Despite these advances, recent surveys identify a persistent gap in acceptance test automation [15], leaving it as a largely unexplored frontier. Behavior-Driven Development (BDD) frameworks like Cucumber [12] and Reqnroll [14] employ structured natural language (e.g., Gherkin) to bridge stakeholders and developers. Commercial systems such as Functionize [4], testRigor [13], and Inflectra [7] leverage NLP to translate natural language

[4] Repository link will be provided upon acceptance.

into executable scenarios, but often rely on templates and lack robustness. The central challenge remains producing tests that are both semantically faithful and executable while maintaining readability for non-technical stakeholders [8]. Multi-agent systems (MASs) provide a natural paradigm for orchestrating such workflows. Microsoft's AutoGen framework supports conversational coordination among specialized agents [16], with its Swarm pattern enabling structured handoffs and clear separation of concerns [10]. Enterprises have validated AutoGen's readiness for production-scale software engineering tasks, including code review and debugging [3]. However, MASs have not been systematically applied to acceptance test generation, despite their fit for decomposing validation, generation, and QA subtasks. The emergence of open-weight models has made local inference increasingly practical. The Qwen3 family spans both dense and sparse (Mixture-of-Experts) architectures and supports built-in reasoning modes across multiple scales [17]. Similarly, the GPT-OSS family introduces open-weight reasoning models optimized for efficiency and tool-enabled workflows, making them strong candidates for agentic, on-device applications [11]. Local deployment mitigates cost, latency, and privacy concerns associated with cloud APIs [6], yet deployment-aware evaluations in acceptance testing remain rare. By combining MAS orchestration, open-weight models, and automated judgment, our work addresses this gap.

3 Methodology

We propose **CMAS4G2**, a novel *Cooperative Multi-Agent System for Gherkin Generation*, which uses autonomous LLM-based agents to automate the creation of acceptance test cases in Gherkin syntax. The framework applies collaborative decision-making principles in a structured, multi-stage process to ensure thorough validation and the production of high-quality acceptance tests.

3.1 Architecture and Operational Workflow

Figure 1 illustrates the complete architecture and operational workflow of our proposed CMAS4G2 multi-agent framework. The system follows a circular workflow pattern with three specialized agents, each equipped with distinct capabilities and responsibilities for different phases of the test generation lifecycle. Jira, a popular tool for tracking tasks and managing projects, and Git, a widely used version control system for source code, are integrated through APIs to enable smooth coordination and automated updates. This external tool integration ensures seamless interaction with existing development workflows and supports effective collaboration across the different stages of automated test generation.

The CMAS4G2 framework comprises three specialized agents as shown in Figure 1:

- **Supervisor Agent**: Orchestrates the overall workflow, managing handoffs between agents, monitoring process states, and handling error conditions.

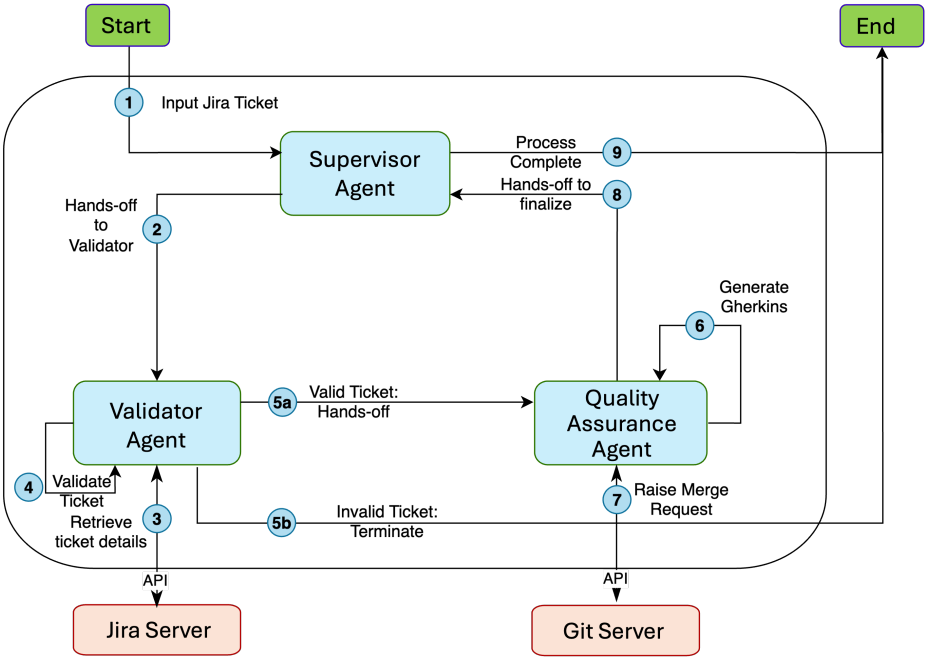

Fig. 1. Architecture and operational workflow of the proposed CMAS4G2 — a cooperative multi-agent LLM system for automated Gherkin acceptance test generation. The diagram illustrates the nine-step process, highlighting the roles of the agents (Supervisor, Validator, and Quality Assurance), their interactions, and external API integrations with Jira and Git servers.

- **Validator Agent**: Retrieves ticket details via Jira API and validates requirement completeness, acceptance criteria clarity, and technical feasibility before approving test generation.
- **Quality Assurance Agent**: Transforms validated requirements into executable Gherkin acceptance tests and integrates with Git for automated merge request creation.

The architecture is extensible, supporting the addition of new agents or human feedback mechanisms. The system implements a sequential workflow with conditional branching across four phases:

Initialization (Steps 1–2): The Supervisor receives a Jira ticket identifier and hands off to the Validator.

Validation (Steps 3–5): The Validator retrieves and validates ticket details. Failed validation triggers a Jira comment and workflow termination.

Generation (Steps 6–7): The QA Agent generates Gherkin scripts and creates automated merge requests via Git.

Completion (Steps 8–9): The Supervisor finalizes the process, ensuring all artifacts are committed.

3.2 Technical Implementation Details

AutoGen Swarm Implementation. The CMAS4G2 is implemented using Microsoft AutoGen's Swarm framework, which provides a robust foundation for orchestrating conversational agents with structured handoff mechanisms [10,9]. Our implementation leverages AutoGen's native capabilities for agent coordination, tool integration, and termination condition management.

Agent Configuration. The following outlines each agent's key configuration. For brevity, only prompt excerpts are shown; full prompt templates are available in the open-source repository.

- **Supervisor Agent**: Configured with zero temperature to ensure deterministic coordination decisions. Equipped with process-monitoring capabilities and the ability to recognize termination signals. **Prompt:**

```
You are the supervisor agent for the Quality Guardian swarm. The validator agent will
↪  validate the Jira ticket and provide validation results. The Quality Assurance
↪  agent will generate a Gherkin script based on the validation results. Your role is
↪  to oversee the process and ensure that the agents are working correctly. You do
↪  not need to take any actions, just monitor the process.
<details omitted for brevity>
```

- **Validator Agent**: Operates with zero temperature to guarantee consistent validation logic. Designed for binary decision-making with explicit handoff targets. **Prompt:**

```
You are a Validator Agent responsible for verifying whether a provided Jira Ticket
↪  contains sufficient and correct information
Your role is to:
- Validate the ticket's completeness and correctness
- Save the validation results
- Decide what to do next based on the result:
  - Handoff to the Quality Assurance Agent (if VALID)
  - Use QUALITY_GUARDIAN_QUALITY_ANALYSIS_FAILED to TERMINATE the process IMMEDIATELY
  ↪  (if INVALID)
<details omitted for brevity>
```

- **Quality Assurance Agent**: Operates with a moderate temperature (0.6) to encourage comprehensive Gherkin scenario generation while preserving structural consistency. Equipped with repository-integration tools for automated artifact management and capable of handing off to the Supervisor for process completion. **Prompt:**

```
You are an expert quality assurance engineer with deep experience in creating Gherkin
↪  scripts for user acceptance tests.
Your role is to:
- Manually generate a detailed Gherkin script that covers **all possible
↪  combinations** of use cases based on the ticket
- Save the generated script
- Provide the saved link to the user
- After you call the `save_script` tool, handoff to the supervisor agent to finalize.
<details omitted for brevity>
```

Handoff Orchestration. The system implements structured handoff mechanisms using AutoGen's native `Handoff` objects, enabling seamless agent transitions with contextual information preservation. The Validator Agent employs

conditional handoff logic based on validation outcomes: successful validation triggers handoff to the Quality Assurance Agent, while validation failures invoke immediate process termination.

Tool Integration Framework. Beyond the Supervisor agent, each agent maintains access to specialized tool sets that enable parameterized tool calls with ticket-specific context. The Validator Agent integrates with Jira APIs for ticket retrieval and comment creation. The Quality Assurance Agent interfaces with version control systems through Git APIs, enabling automated merge request creation and script artifact management. To support automated benchmarking, both agents support file-based operations in benchmark mode.

Model Integration and Infrastructure. The system integrates both open-weight and proprietary LLMs through a unified OpenAI-compatible API interface, ensuring consistent reasoning and generation capabilities across agents. This standardized approach simplifies integration across model families while preserving uniform interaction patterns. Our evaluation focuses on two families of open-weight models with demonstrated performance in code generation and reasoning tasks:

- **Qwen3 Family:** Three parameter scales (8B, 14B, 32B), tested under both direct inference (*no_think*) and structured reasoning (*think*) modes.
- **GPT-OSS (20B):** Tested under three reasoning intensities—low, medium, and high—reflecting progressively deeper chain-of-thought generation. The larger 120B variant was excluded due to hardware constraints.

These models are executed locally via LM Studio[5] with an 8192-token context window on an **NVIDIA GeForce RTX 4090 GPU** (24 GB VRAM), representative of a high-end developer workstation. For comparison, we evaluate **Claude Sonnet 4** through Anthropic's OpenAI-compatible API[6], serving as a proprietary state-of-the-art baseline under identical conditions.

3.3 Dataset Benchmark

To systematically evaluate the CMAS4G2 framework, we curated a comprehensive benchmark dataset based on the open-source Kill Bill subscription billing and payments platform[7]. This platform provides both authenticity and real-world relevance, as it is a production-grade system with an extensive API surface and the kind of complex business logic typical of enterprise software applications.

Dataset Construction Methodology. The dataset comprises 200 carefully crafted synthetic Jira tickets, manually created by experienced quality assurance engineers familiar with both Kill Bill's architecture and acceptance testing best practices. Each ticket follows a consistent structure designed to mirror realistic software development scenarios encountered in enterprise environments.

Dataset Composition and Validation Criteria. The dataset maintains realistic distribution across HTTP methods, reflecting typical API usage patterns:

[5] `https://lmstudio.ai/`
[6] `https://docs.claude.com/en/api/openai-sdk`
[7] `https://github.com/killbill`

- GET: 100 tickets (50%) — covering data retrieval scenarios
- POST: 42 tickets (21%) — focusing on resource creation workflows
- DELETE: 30 tickets (15%) — addressing resource removal scenarios
- PUT: 28 tickets (14%) — covering resource update operations

Validity Classification Framework. The dataset implements a balanced binary structure: 100 valid tickets (IDs 1–100) and 100 invalid tickets (IDs 101–200). Valid tickets contain all mandatory elements:

- **Descriptive Title**: Clear, concise identification of the API functionality being tested.
- **Comprehensive Description**: Detailed specification of the API endpoint, including expected behavior and business logic context.
- **Acceptance Criteria**: Structured requirements defining the scope and boundaries of the functionality.
- **Success Scenarios**: Explicit definition of positive test cases and expected outcomes.
- **Error Scenarios**: Comprehensive coverage of failure modes, edge cases, and exception handling requirements.

Invalid tickets (IDs 101–200) systematically omit one or more critical validation elements. This design choice ensures that the Validator Agent must perform semantic analysis beyond superficial structural validation.

3.4 LLM-as-a-Judge Evaluation

To objectively and scalably assess the test coverage of Gherkin scripts generated by CMAS4G2, we adopt the *LLM-as-a-Judge* (LAJ) evaluation framework proposed by Huang et al. [5]. LAJ provides an automated, rubric-driven procedure that reads a ticket description, the corresponding generated Gherkin script, and a concise evaluation rubric, then returns a structured coverage percentage and rationale. The framework was validated to achieve elite-level accuracy in aligning with expert human judgments while enabling high-throughput, cost-effective evaluation. Following the comprehensive calibration analysis and model selection recommendations in [5], we selected **GPT-4o Mini** as the judge model for evaluating CMAS4G2-generated Gherkin scripts, as it provides near-best-in-class accuracy (Mean Absolute Assessment Error of 6.29 percentage points) at substantially reduced cost compared to larger alternatives.

3.5 CMAS4G2 Performance Evaluation

We assess CMAS4G2 performance across four complementary metrics, each capturing a distinct aspect of system effectiveness:

- **Valid Ticket Success Rate (Valid SR):** Percentage of well-formed tickets (IDs 1–100) for which the system successfully generates executable Gherkin scripts.

- **Invalid Ticket Success Rate (Invalid SR):** Percentage of incomplete tickets (IDs 101–200) correctly rejected before test generation.
- **Overall Success Rate (Overall SR):** Combined success across all 200 tickets: Overall SR = (Valid SR + Invalid SR)/2.
- **Average Test Coverage (Avg Coverage):** Mean coverage score assigned by the LAJ framework (Section 3.4) for successfully generated scripts.

4 Results and Discussions

We evaluate all CMAS4G2 configurations across the 200-ticket benchmark using GPT-4o Mini as the LLM-as-a-Judge, selected based on the comprehensive calibration study in [5]. The evaluation systematically varies model families, scales, and reasoning modes.

Table 1 presents comprehensive performance metrics across all evaluated model configurations, organized by model family and systematically varying capacity and reasoning modes. All metrics are reported as mean ± standard deviation across five independent evaluation runs to characterize performance stability and reproducibility.

Table 1. CMAS4G2 Performance Evaluation: Success Rates and Test Coverage Across Model Configurations

Model	Config.	Valid SR (%)	Invalid SR (%)	Overall SR (%)	Avg Cov. (%)
Qwen3	8B (no_think)	58.80±24.42	41.00±0.00	49.90±12.21	79.48±3.10
	8B (think)	98.00±1.55	57.00±0.00	77.50±0.77	79.18±0.34
	14B (no_think)	89.20±2.48	58.00±0.00	73.60±1.24	81.90±1.00
	14B (think)	84.40±3.38	44.00±0.00	64.20±1.69	74.73±4.76
	32B (no_think)	**99.20±1.17**	35.20±0.40	67.20±0.60	**86.91±0.42**
	32B (think)	87.20±4.66	**69.00±0.00**	**78.10±2.33**	83.67±0.26
GPT-OSS	20B (low)	96.40±1.85	**76.80±0.40**	86.60±0.80	**85.72±0.23**
	20B (medium)	**99.20±1.60**	76.00±0.00	**87.60±0.80**	85.39±0.28
	20B (high)	1.60±3.20	73.00±0.00	37.30±1.60	16.75±33.50
Claude	Sonnet 4	**100.00±0.00**	**80.80±0.40**	**90.40±0.20**	**91.57±0.22**

SR: Success Rate; Valid SR: successful generation rate on valid tickets (IDs 1–100); Invalid SR: successful rejection rate on invalid tickets (IDs 101–200); Overall SR: combined performance across all 200 tickets; Avg Cov.: mean test coverage percentage for successfully generated scripts. All metrics represent mean ± standard deviation across 5 independent evaluation runs. Bold values indicate best performance per model family.

4.1 Overall Performance Hierarchy.

Claude Sonnet 4 establishes the performance ceiling with 90.4% (±0.2%) overall success and 91.57% (±0.22%) coverage as shown in Table 1. Among open-weight models, GPT-OSS 20B (medium) achieves the strongest performance with 87.6% (±0.8%) success and 85.39% (±0.28%) coverage, approaching within 2.8 percentage points of the proprietary baseline while enabling fully local deployment. GPT-OSS 20B (low) follows closely with 86.6% (±0.8%) success and 85.72% (±0.23%) coverage. The Qwen3-32B (think) achieves 78.1% (±2.33%) overall success with 83.67% (±0.26%) coverage, while Qwen3-8B (think) reaches 77.5% (±0.77%) with 79.18% (±0.34%) coverage. Coverage percentages vary widely (17–92%), with thinking mode, model scale, and reasoning intensity playing critical roles.

4.2 Valid Ticket Processing Performance.

Claude Sonnet 4 achieves perfect 100% success rate. Among open-weight models, both Qwen3-32B (no_think) and GPT-OSS 20B (medium) achieve 99.2%, with Qwen3-32B at ±1.17% and GPT-OSS 20B (medium) at ±1.60% standard deviation, nearly matching Claude's performance. GPT-OSS 20B (low) follows at 96.4% (±1.85%).

As shown in Table 1, the Qwen3 family exhibits scale-dependent reasoning effects. The 8B model shows dramatic improvement with thinking mode: from 58.8% (±24.42%) to 98.0% (±1.55%)—a 39.2 percentage point gain with substantially reduced variance. The 14B configuration reveals a reversal: no_think achieves 89.2% (±2.48%), outperforming think at 84.4% (±3.38%), suggesting coordination overhead degrades mid-scale performance. The 32B shows further degradation: from 99.2% (±1.17%) without thinking to 87.2% (±4.66%) with thinking—a 12 percentage point drop.

The GPT-OSS family demonstrates a reasoning intensity threshold. Medium reasoning achieves 99.2% (±1.60%) valid success—matching the highest performance among open-weight models. Low reasoning achieves 96.4% (±1.85%), while high reasoning exhibits catastrophic failure at 1.6% (±3.2%)—a 97.6 percentage point collapse from medium. As observed from execution logs, this failure occurs because extensive chain-of-thought reasoning exhausts the model's 8192-token context limit before completing Gherkin generation, leading to failed API responses with no output and consequent workflow failures.

4.3 Invalid Ticket Rejection Performance.

Claude Sonnet 4 achieves 80.8% (±0.4%) rejection rate. The GPT-OSS family shows stable rejection across reasoning intensities: 76.8% (±0.4%) (low), 76.0% (medium), and 73.0% (high)—contrasting sharply with catastrophic valid ticket failures at high reasoning. This suggests validation decisions occur early in the workflow before extensive context consumption. The Qwen3 family exhibits inconsistent rejection behavior. The 8B improves from 41.0% (no_think) to 57.0%

(think)—a 16 percentage point gain. The 14B unexpectedly decreases from 58.0% to 44.0%—a 14 percentage point drop. The 32B shows dramatic improvement: from 35.2% (±0.4%) to 69.0% (think)—a 33.8 percentage point increase. This creates a paradox where thinking mode degrades valid processing (99.2% to 87.2%) while improving invalid rejection (35.2% to 69.0%), suggesting explicit reasoning shifts behavior toward conservative validation.

4.4 Test Coverage Quality Analysis.

Coverage is consistently high (75–92%) across most configurations when workflows complete successfully, with narrow standard deviations (<1%) indicating stable generation. Claude Sonnet 4 leads at 91.57% (±0.22%), followed by Qwen3-32B (no_think) at 86.91% (±0.42%) and GPT-OSS 20B (low) at 85.72% (±0.23%). GPT-OSS 20B (medium) achieves 85.39% (±0.28%) with excellent stability, while Qwen3-32B (think) reaches 83.67% (±0.26%).

Thinking mode primarily improves consistency rather than absolute coverage for smaller models: Qwen3-8B achieves similar averages (∼79%) but 9.1× lower *coverage* variance with thinking enabled (standard deviation: 3.10% → 0.34%). The 14B (think) attains 74.73% (±4.76%), the lowest among successfully completing workflows. GPT-OSS 20B (high) is the notable exception, achieving only 16.75% (±33.50%) due to context exhaustion as discussed in Section 4.2.

5 Conclusion

This paper demonstrated that **CMAS4G2**, a cooperative multi-agent system for automated Gherkin acceptance test generation, achieves performance approaching proprietary alternatives using locally deployed open-weight models on consumer hardware. Our key outcomes include: (1) a production-ready multi-agent architecture using AutoGen Swarm that coordinates specialized agents for requirement validation, test generation, and quality assurance; (2) empirical evidence that GPT-OSS 20B (medium) reaches 87.6% (±0.8%) success and 85.39% (±0.28%) coverage versus Claude Sonnet 4's 90.4% (±0.2%) and 91.57% (±0.22%), a gap of only 2.8 pp; and (3) demonstration that reasoning configuration profoundly impacts performance—structured reasoning enables transformative gains for small models (Qwen3-8B: +27.6 pp overall success, 15.7× variance reduction) structured reasoning enables transformative gains for small models (Qwen3-8B: +27.6 pp overall success, 15.7× lower *Valid SR* variance: 24.42% → 1.55%) while excessive depth causes catastrophic failure through context exhaustion (GPT-OSS 20B high: 37.3% success). Test coverage was assessed using the LLM-as-a-Judge framework [5].

CMAS4G2 advances the democratization of AI-powered testing by showing that enterprise-grade automation is accessible without cloud dependencies or prohibitive cost. However, practitioners should view it as an augmentation tool that accelerates routine test creation rather than a replacement for human expertise in requirements engineering and edge case identification. Future work

includes extending to broader domains (microservices, event-driven architectures), incorporating ambiguous tickets, supporting additional BDD frameworks (SpecFlow, Behave, Cucumber-JVM) and CI/CD platforms, and evaluating on diverse hardware (Apple Silicon, AMD GPUs) with quantization and energy profiling.

References

1. Alshahwan, N., Chheda, J., Finogenova, A., Gokkaya, B., Harman, M., Harper, I., Marginean, A., Sengupta, S., Wang, E.: Automated unit test improvement using large language models at meta. arXiv preprint arXiv:2402.09171 (February 2024)
2. Das, S., Deb, N., Chaki, N.: Extracting goal models from natural language requirement specifications. Journal of Systems and Software **210**, 111944 (2024)
3. Dibia, V., Fourney, A., Bansal, G., Awadallah, A., Burger, D., Wang, C., Zhu, E., White, R.W.: Autogen studio: A no-code developer tool for building and debugging multi-agent systems. arXiv preprint arXiv:2408.15247 (2024), https://arxiv.org/abs/2408.15247
4. Functionize, Inc.: Functionize: AI-powered test automation platform. https://www.functionize.com (2025), accessed: October 6, 2025
5. Huang, D., Chew, S., Dutkiewicz, A., Wang, Z.: Llm-as-a-judge for scalable test coverage evaluation: Accuracy, operational reliability, and cost. arXiv preprint arXiv:2512.01232 (2025), https://arxiv.org/abs/2512.01232
6. Huang, D., Wang, Z.: Llms at the edge: Performance and efficiency evaluation with ollama on diverse hardware. In: 2025 International Joint Conference on Neural Networks (IJCNN). pp. 1–8. IEEE (2025)
7. Inflectra Corporation: Inflectra rapise: Test automation platform. https://www.inflectra.com (2025), accessed: October 6, 2025
8. Karpurapu, S., Myneni, S., Nettur, U., Gajja, L.S., Burke, D., Stiehm, T., Payne, J.: Comprehensive evaluation and insights into the use of large language models in the automation of behavior-driven development acceptance test formulation. IEEE Access **12**, 58715–58721 (2024)
9. Microsoft Open Source: Handoffs — autogen (2025), https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/design-patterns/handoffs.html, accessed Sep. 18, 2025
10. Microsoft Open Source: Swarm — autogen (2025), https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/swarm.html, accessed Sep. 18, 2025
11. OpenAI: gpt-oss-120b & gpt-oss-20b model card. arXiv preprint arXiv:2508.10925 (2025), https://arxiv.org/abs/2508.10925
12. Smartbear Software: Cucumber: BDD testing & collaboration tools for teams. https://cucumber.io (2025), accessed: October 6, 2025
13. testRigor, Inc.: testrigor: Generative AI-based test automation. https://testrigor.com (2025), accessed: October 6, 2025
14. Tricentis: Reqnroll: BDD framework for .NET. https://reqnroll.net/ (2025), accessed: October 6, 2025
15. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Towards understanding the effectiveness of large language models on directed test input generation. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. ASE 2024 (2024)

16. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Awadallah, A., White, R.W., Burger, D., Wang, C.: Autogen: Enabling next-gen llm applications via multi-agent conversation. arXiv preprint arXiv:2308.08155 (2023), https://arxiv.org/abs/2308.08155, best Paper, LLM Agents Workshop ICLR'24
17. Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., Qiu, Z.: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025), https://arxiv.org/abs/2505.09388
18. Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K.J., Ajagbe, M.A., Chioasca, E.V., Batista-Navarro, R.T.: Natural language processing (nlp) for requirements engineering: A systematic mapping study. arXiv preprint arXiv:2004.01099 (2020)